FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Jaroslav Švarc

Data-driven low-code programming system

Department of Distributed and Dependable Systems

Supervisor of the bachelor thesis: Mgr. Tomáš Petříček, Ph.D.
Study programme: Computer science

Prague 2025

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Data-driven low-code programming system

Author: Jaroslav Švarc

Department: Department of Distributed and Dependable Systems

Supervisor: Mgr. Tomáš Petříček, Ph.D., Department of Distributed and Dependable Systems

Abstract: Abstract

Keywords: low-code programming, programming systems, data-driven programming

Název práce: Data-driven low-code programming system

Autor: Jaroslav Švarc

Katedra: Katedra distribuovaných a spolehlivých systémů

Vedoucí bakalářské práce: Mgr. Tomáš Petříček, Ph.D., Katedra distribuovaných a spolehlivých systémů

Abstrakt: Abstract

Klíčová slova: low-code programování, vývojové nástroje, datově řízené programování

Contents

Introduction **7**

1 Background **10**
1.1 Low-code development systems . . . 10
1.1.1 Hypercard . . . 10
1.1.2 Darklang . . . 12
1.1.3 Sketch-and-Sketch - Output-directed programming . . . 13
1.2 Web Development . . . 14
1.2.1 The JavaScript Object Notation format . . . 14
1.2.2 Elm . . . 15
1.2.3 The Elmish architecture . . . 15
1.2.4 F# and Fable . . . 16
1.2.5 React and Feliz . . . 16

2 Design principles and Requirements **18**
2.1 Data-driven approach . . . 18
2.2 Low-code editor . . . 19
2.3 Code generation . . . 19
2.4 Requirements . . . 20
2.4.1 Functional Requirements . . . 20
2.4.2 Non-functional Requirements . . . 20

3 Core Logic - Domain and Operations **21**
3.1 Hole-based Approach . . . 21
3.2 Domain model and Mapping . . . 21
3.2.1 JSON Type . . . 21
3.2.2 RenderingCode Type . . . 22
3.2.3 Data Mapping: JSON to RenderingCode . . . 23
3.3 Incremental Creation Process . . . 24
3.3.1 Traversal . . . 24
3.3.2 Hole Replacement and Modification . . . 25
3.3.3 Creation of a single RenderingCode . . . 26
3.3.4 Combining RenderingCode and JSON data . . . 26
3.4 Summary . . . 27

4 Implementation **28**
4.1 Technologies . . . 28
4.1.1 Alternatives . . . 28
4.2 System architecture . . . 29
4.2.1 Core Logic Module . . . 30
4.2.2 Editor module . . . 30
4.3 Main Features . . . 32
4.3.1 User interface implementation . . . 32
4.3.2 User Interface layout . . . 33
4.3.3 Custom Rendering . . . 33

4.3.4 Code generation . . . 34
4.4 Building and Deployment . . . 35
4.4.1 Build Tools and Libraries . . . 35
4.4.2 Build targets . . . 35
4.4.3 Deployment . . . 36
4.5 Testing . . . 36
4.6 Summary . . . 36

5 Benchmarks **38**
5.1 Methodology . . . 38
5.2 To-do list application . . . 38
5.3 Counter Task (7GUIs) . . . 38
5.3.1 Creation process . . . 38
5.3.2 Results . . . 39
5.3.3 Analysis . . . 39
5.4 Temperature Converter Task (7GUIs) . . . 40
5.5 Evaluation . . . 40

6 Discussion **41**
6.1 Evaluation of the approach . . . 41
6.1.1 Benefits of the approach . . . 41
6.1.2 Limitations of the approach . . . 42
6.2 Evaluation of the implementation . . . 43
6.2.1 Positive aspects of the implementation . . . 43
6.2.2 Limitations of the implementation . . . 43
6.3 Future work . . . 44

Conclusion **46**

Bibliography **47**

List of Figures **49**

Introduction

Since the advent of the *World Wide Web*, developers have been creating web applications mainly by writing and modifying text-based code. Today, this process is made more accessible by using different tools, libraries, frameworks, and programming systems. For example, tools such as the *React* [1] library or the *Vue.js* [2] framework make it possible to compose web applications from reusable components but still require the user to write and modify text-based code.

Some programming systems allow an alternative approach of *low-code* software development. Pinho et al. [3] define *low-code programming systems* as "systems that aim to significantly reduce the amount of code needed to create specific applications." These systems employ an alternative way of creating and modifying the software components by providing a *Graphical User interface (GUI)* and modification tools that aid the user during the creation process. The popularity of these systems has been increasing in recent years, as stated by Sahay et al. [4].

Sahay et al. [4] identify that low-code systems provide two main approaches to creating software applications, and they are **UI to Data** and **Data to UI**. *UI to Data* involves creating the application's UI and populating it with data, whereas *Data to UI* is a data-driven approach where the UI is created *after* modeling necessary data. Systems such as *Mendix* [5] or *Darklang* [6] specialize in providing a comprehensive low-code development platform for creating specific components of web applications. Other programming systems, such as *Sketch-n-Sketch* presented by Hempel et al. [7], combine the low-code approach with other programming techniques, such as *output-directed programming* [8], to enhance the system's functionality and development process.

This thesis will focus mainly on the *data to UI* low-code approach. We aim to explore the *data to UI* approach in the context of creating *client-side web applications*, providing an alternative to traditional text-based development methods. We will design and implement a *prototype* low-code programming system named **InterfaceSmith**. The *data to UI* process will be influenced by the *type* of data provided to the system and also its *structure*. The application's UI elements will then be created through a sequence of steps, mirroring the structure of the provided data. The programming system will then aid the developer in modifying the user interface by providing relevant context menus. After creating and modifying the UI elements, the system will provide behavior modification options inspired by *Elm* [9] and the Elmish architecture[10]. After the developer finishes the implementation process, they can preview the created application and download its text-based representation.

To evaluate the presented system, we will *benchmark* the *InterfaceSmith* prototype system on several different tasks. Based on the definition of low-code programming systems, we will evaluate the results based on the number of lines of code (LOC) that the user needs to write to implement the selected tasks and whether the functionality of the desired task has been successfully implemented.

The main output of this thesis is the InterfaceSmith application shown in Figure 1. The user interface is inspired by *Darklang* [6] and consists of a movable canvas containing various draggable elements. The main element is the UI modification and preview element, which shows a preview of the application's created parts alongside modification menus and is also resizable, simulating a browser window. Other elements

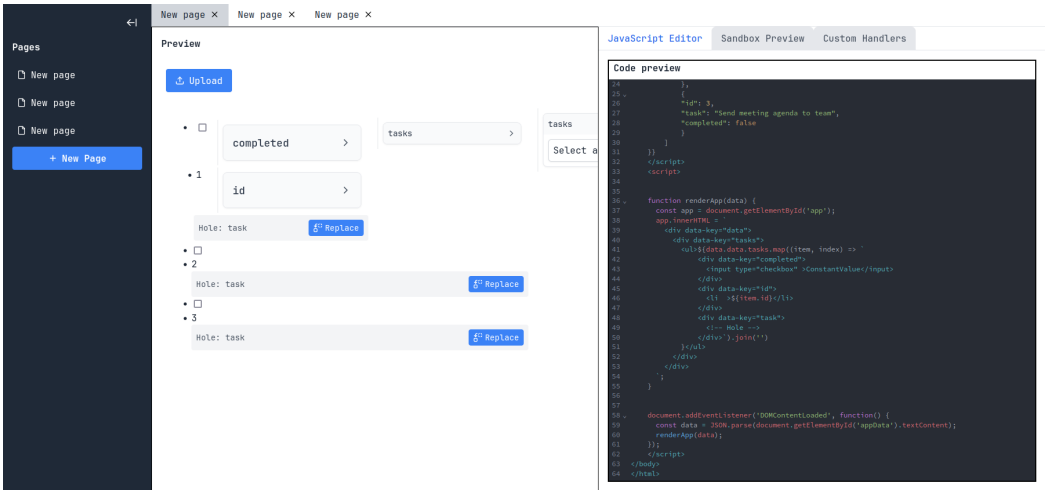

Figure 1 InterfaceSmith's UI example

include a view of the uploaded data, elements for creating and modifying the behavior of the UI elements, and elements for creating handler JS functions. The resulting code is generated based on the created elements. The system creates an Elmish-like application in pure JavaScript, but it could be extended to target other technologies.

Goals

The main goals of this thesis are the following:

1. Explore the applicability of the low-code programming system employing the data-driven approach to the creation of UI elements of web applications:
    - Define the *data-driven* approach.
    - Define the approach in the context of creating web applications based on concrete data.
    - Describe the design principles behind the approach.
    - Present a description of creating UI elements based on the input data.
    - Evaluate the feasibility of this approach in the context of the prototype application.
2. Create a working **prototype programming system** implementing the data-driven approach according to the requirements described in Chapter 2.
3. Benchmark the prototype application on the following tasks:
    - A simple **To-do list** application inspired by the *TodoMVC* [11].
    - **Counter** task from the *7GUIs benchmark* [12].
    - **Temperature converter** task from the *7GUIs benchmark* [12].

Contributions and Outline

This thesis contributes to the field of software development, specifically web development systems employing a data-driven low-code approach.

- In Chapter 2, we describe the main design principles that influenced the design of the proposed programming system and the functional and non-functional requirements for the *InterfaceSmith* prototype application.
- In Chapter 3, we describe the core logic of our programming system. We introduce the data-driven hole-based approach to creating UI elements, describe the domain model and essential operations, and describe the incremental creation process.
- In Chapter 4, we describe the implementation specifics of the *InterfaceSmith* application, including its architecture, technologies used, and the design of its user interface.
- In Chapter 5, we showcase the creation of a user interface for a simple to-do list application and several other tasks from *7GUIs benchmark* [12].
- In Chapter 6, we discuss the benefits and limitations of our proposed programming system, and we evaluate our *InterfaceSmith* application. We also provide several future avenues of research.

1 Background

1.1 Low-code development systems

In the previous chapter, we briefly described that our thesis aims to create a *low-code data-driven* programming system. In this section, we describe several existing systems that employ the low-code development approach to understand better how our resulting application fits into the low-code programming system landscape. The low-code development approach can be defined as follows:

Definition 1 (Low-code development approach)**.** *Defined by Pinho et al. [3] as "An approach for software development that uses tools that minimize (or eliminate) the number of lines of code written."*

This definition accommodates a broad spectrum of tools and development systems. The main category of these tools is *visual programming tools* such as *Darklang* [6] or *Mendix* [5]. The visual programming tools provide a Graphical User Interface (GUI), which allows users to create and modify software by interacting with the interface. They differ in the style of UI, provided functionality, their intended use case, and other features, such as automatic deployment of the created software.

The next category of low-code development tools is *Integrated development environments (IDEs)* with code generation capabilities. For example, *JetBrains Rider* [13] provides advanced context-aware code auto-completion alongside tools to generate boilerplate code and project scaffolding. Code generation can also be provided by integrating *Large Language Models (LLMs)* into existing IDEs, such as the *Github Copilot* [14] extension for the *Visual Studio Code* [15] editor.

The last large category of low-code development tools is *Static site generators (SSGs)*, which transform files written in a simple markup language into a static website. A popular example of SSGs is the *Hugo* [16] static site generator, which also provides tools for customizing the transformation process and supports custom templating.

To better understand the *InterfaceSmith* system, we will describe three programming systems that greatly influenced its design. We explain how these particular systems inspired the design of our system in Chapter 2. The *Hypercard*, *Darklang*, and *Sketch-and-Sketch* are three different systems belonging to the visual programming category of tools. Each system has a different intended use case, User Interface, and capabilities.

1.1.1 Hypercard

Hypercard is a low-code development system created by Bill Atkinson for the Macintosh operating system. Apple released the program in 1987 at the Macworld exposition in Boston [17]. Apple developed and maintained the program until 1998. The popularity of the program ensured that similar programs and clones of Hypercard were created after its discontinuation.

The following summary of functionality is based on a manual for the Hypercard system written by Goodman [18]. The fundamental elements that the user creates are called *cards*. Cards can hold data as text, have custom graphics, contain buttons, and implement custom behavior. Users can implement the custom behavior using a

scripting language called *HyperTalk*. Then, users can group cards into *stacks*. A stack is a collection of cards with the same type of information. The program saves a stack as a single file to the disk. Finally, users can distribute and modify these stacks. Creation and modification of the card is done through the low-code interface. The program provides several options for the different card elements.

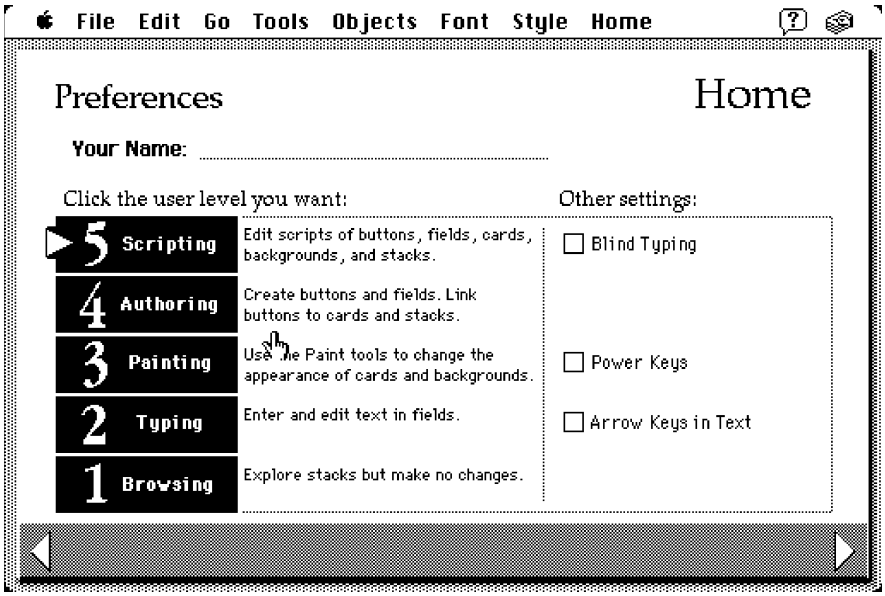

Figure 1.1 Selection of a user level in the Hypercard preferences menu.

The program offers users a choice of a user level, as seen in figure 1.1. The program changes the user interface's capabilities based on the selected user level. The higher the selected user level, the more options the program enables and displays. A selected level enables all previous-level options alongside other options and functionality. Users can choose from five different user levels:

1. **Browsing:** Enables no modification of cards and is mainly intended to be used to view the different cards inside a stack.

2. **Typing:** Enables inserting and modifying text data inside the cards.

3. **Painting:** Enables the creation of custom graphics inside the cards. The program provides different graphical options and tools.

4. **Authoring:** Adds the ability to add buttons and fields. This way, the users can add card functionality without writing code.

5. **Scripting:** Adds the ability to use the HyperTalk scripting language to modify the behavior of the different elements inside the cards.

This allows the system to be adopted by a wide range of users, as less experienced users can learn to use the interface more easily and gradually increase their proficiency with the program without being overwhelmed by options. Advanced users can set the highest user level and use the program in its entirety from the start.

1.1.2 Darklang

Darklang [6] is a cloud-based low-code programming system for building web application backends created by Paul Biggar. The original system's development was discontinued in 2023 and renamed Darklang-classic.

The following summary follows the official Darklang documentation[1]. The Darklang project consists of a low-code editor and a programming language. It also provides tools for creating persistent storage and for deployment. The editor provides the service's main functionality. It consists of a canvas that displays the created application elements as draggable boxes as seen in Figure 1.2. Each box provides options to modify the element. These options include different menus and input fields. Some elements can be modified using the Darklang programming language to implement custom behavior. The user can create program elements of the following categories:

- HTTP handlers - definition of API endpoints
- Persistent storage - database creation and modification
- Workers - processing of background tasks
- Cron jobs - scheduled jobs with custom behavior
- REPL - a general-purpose programmable element
- Functions - a reusable element with custom behavior

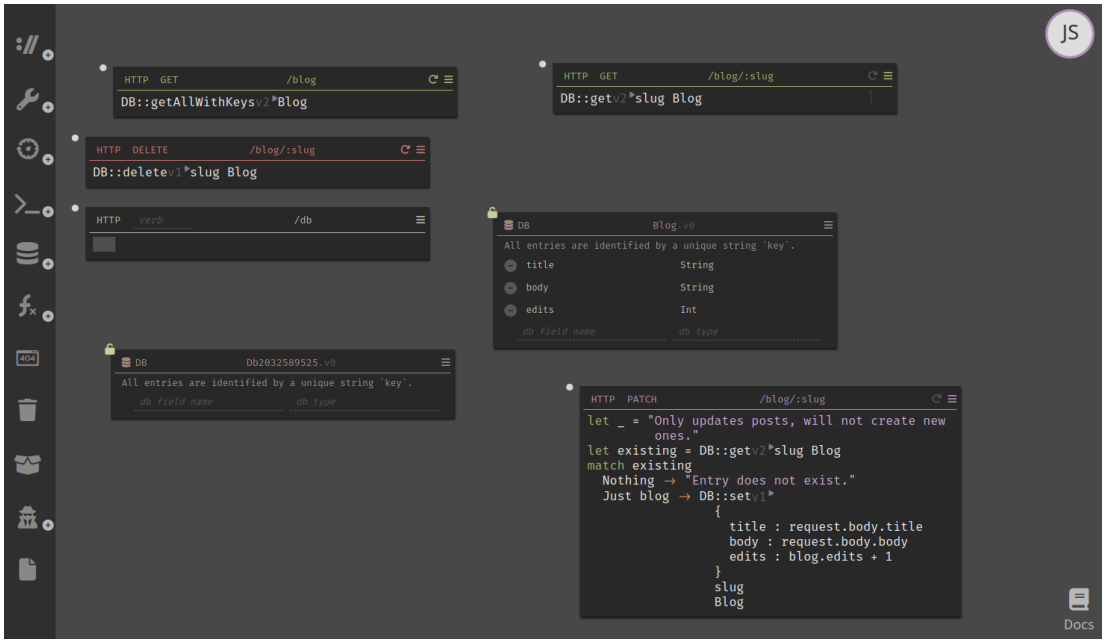

Figure 1.2 Example of Darklang's drag-and-drop user interface.

By combining these elements, the users can create application backends of varying complexity. The system automatically redeploys the entire application after the user creates and modifies a particular element. This allows for an interactive style of development where changes made to the system are almost immediately visible.

[1] `https://docs.darklang.com/` [visited on 2024-04-22][online]

1.1.3 Sketch-and-Sketch - Output-directed programming

Output-directed programming is described by Hempel et al. [7] as a programming paradigm where users construct plain text programs using mouse-based operations on the program's output. The difference between this paradigm and other low-code programming approaches is that the source code is still the main representation of the program. When making a change using the mouse-based operations is too hard or impossible, the user can still change the output by directly changing the source code. This approach is made possible thanks to the use of *live synchronization* and *trace-based program synthesis* as defined by Chugh et al. [8]

One example of a programming system that demonstrates this paradigm is called *Sketch-and-Sketch* [7, 8]. It is a browser-based application that provides a bimodal programming environment. It allows creating and modifying Scalable Vector Graphics (SVG) by directly manipulating the program's SVG output. The application consists mainly of an editor window for the source code, a canvas window displaying the program's output, and a button to run the program. After clicking on the canvas, the output is overlaid by context menus and *widgets*, allowing users to modify the program's *sub-expressions* and *intermediate values* as seen in Figure 1.3.

Sub-expressions refer to a certain syntactic scope of the program and the user can select and modify it. *Intermediate values* are values produced at the intermediate execution steps of the program. The program provides widgets for different types of intermediate values such as *offset*, *point*, or *list* widgets. Widgets can take the form of draggable boxes or input fields.

Another programming system implementing the direct manipulation paradigm is *Transformic* introduced by Schreiber et al. [19]. It is a functional interpretation of the Morphic GUI Framework providing direct-manipulation functionality.

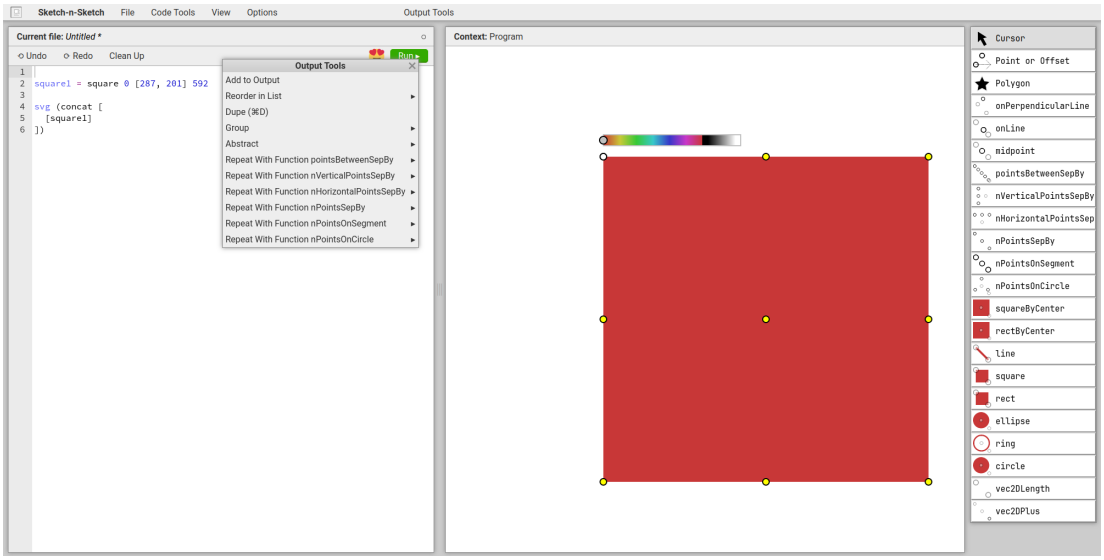

Figure 1.3 Example of Sketch-and-Sketch's user interface.

1.2 Web Development

In the previous section, we described the low-code development approach and several existing low-code development systems. In this section, we introduce web development tools and technologies that we use for the *InterfaceSmith* programming system or that influence its design and implementation.

To create a data-driven programming system, we must select a data representation format that is easy for users to obtain, understand, and modify. We choose the *JavaScript Object Notation format* for our input data representation, as it is widely used in web development, is easy to write and understand, and can be easily parsed by.

Our programming system provides a GUI, and technologies such as the *Elm* [9] functional programming language influenced the nature of our implementation. We do not use Elm directly, but we model our implementation after the *Elmish* [10] architecture, which is a simplified form of the programming model used by Elm.

To implement our programming system, we use the *F#* [20] programming language alongside the *Fable* [21] compiler, which compiles the application into *JavaScript*, which allows the application to run in a browser-based environment.

We use the *Feliz* [22] library to implement the user interface of our programming system. The *Feliz* library provides a *Domain-Specific Language (DSL)* for building *React* user interface components and applications in F#. The *React* [1] library provides tools to create composable components, handle component-level storage and behavior, and handle the re-rendering of specific UI elements.

1.2.1 The JavaScript Object Notation format

The JavaScript Object Notation is defined by Bray [23] as a lightweight, text-based data interchange format. It is language-independent and easy for both humans to read and write and machines to parse and generate. JSON supports the following data types: *Objects*, *Arrays*, *Numbers*, *Strings*, *Booleans*, and the *null* type. We can see an example of the JSON syntax in Figure 1.4

```
1 {
2     "JsonObject" : {},
3     "JsonList": [],
4     "JsonString" : "Content",
5     "JsonNumber": 0,
6     "JsonBool" : true,
7     "JseonNull": null
8 }
```

Figure 1.4 Example of the available JSON data types and the JSON syntax

It is widely used in web development for transmitting data between the server and client-side web applications, or used for configuration of different development tools. Thanks to its popularity, many different parsers and other tools have been created to support its use, such as the *Fable.SimpleJson* [24] library which we use in our implementation.

1.2.2 Elm

Elm [9] is a functional programming language presented by Czaplicki and Chong [25]. It allows the creation of responsive graphical user interfaces by employing the *Functional Reactive Programming (FPR)* approach. The functional reactive programming approach applies pure functional programming paradigms to time-varying values, as introduced by Elliott and Hudak [26]. Time-varying values can represent the input and output of the GUI or other information channels, such as server requests. The applications to time-varying values are known as *signals*. Elm simplifies the approach of functional reactive programming, by assuming that the signals are discrete. This means that it avoids unnecessary recomputation when the signals are unchanged. Discrete change of a signal is called an event. Events trigger the recomputation and the application is updated. Elm also provides tools and abstractions to enable asynchronous computations.

The main categories of GUI elements in Elm are *forms* and *elements*. *Elements* represent a *rectangular* GUI element with properties such as width and height. Elements can contain data in a form of text, video or an image and can be composed together. Forms on the other hand represent two dimensional elements of arbitrary shapes with modifiable properties such as color and texture.

Reactive GUIs can be implemented by combining GUI elements and input signals. Elm provides a range of primitive signals such as ”Mouse.clicks” which triggers on mouse click. Some primitive signals may require arguments for their constructor such as the ”Window.dimensions” signal.

1.2.3 The Elmish architecture

The *Elmish* architecture, or the *Model View Update(MVU)* architecture, is a programming architecture for building web applications. The architecture is inspired by Elm's FPR programming model but was simplified to make the programming model more usable and easier to learn, as stated by Czaplicki [27].

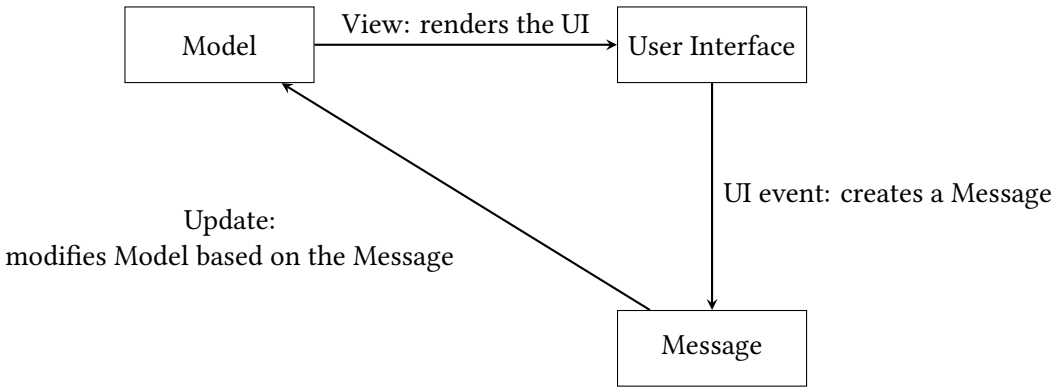

Figure 1.5 Diagram of the Elmish Architecture

In Figure 1.5, we can see a diagram representing the architecture. The architecture consists of the following components:

- **Model:** The *Model* is a data-structure which keeps the state of the entire application. It holds data and other necessary values that the application uses. The data-structure is usually immutable. This means that making changes to the

state results in a creation of an updated copy of the state which becomes the new state.

- **View:** View is a function which takes the current state of the application and creates a graphical user interface. The interface usually consists of multiple GUI elements displaying the application state.
- **Update:** Update is a function which updates the model based on the inputs from the graphical user interface which calls the update function with a certain *Message*. Messages are similar to events in the Elm language. Each message represents a certain change to the model. The update function updates the model based on the provided message and the current application state. The message type is usually represented as a *discriminated union*. This enables the update function to use pattern-matching expressions to efficiently update the model.

1.2.4 F# and Fable

F# [20] is a functional-first programming language running on the .NET platform. Although it is mainly a functional programming language, it allows the use of other paradigms, such as the imperative and object-oriented paradigms. We chose F# to implement our low-code programming system due to its strong type system, functional programming capabilities, and seamless integration with the .NET ecosystem. The reasons for this choice are further elaborated in Chapter 4.

Fable [21] is a compiler capable of compiling programs written in F# to multiple target languages. One of the available target languages is JavaScript, and the compiler produces human-readable and formatted JavaScript code. This enables the creation of web applications in a type-safe functional programming language while also enabling the use of tools and libraries from the wider JavaScript ecosystem.

The Fable compiler supports the use of some types from .NET Base Class and FSharp.Core libraries [28] and attempts to compile these types to native JavaScript whenever possible. This feature allows developers to leverage their existing knowledge of F# and .NET while building web applications.

By using F# with Fable, we can develop our low-code programming system using a statically-typed functional language while targeting the JavaScript runtime environment of web browsers, and also allowing us to use tools from the wider JavaScript ecosystem.

1.2.5 React and Feliz

The *React* [1] JavaScript library, initially developed by *Facebook*, is used to create user interfaces for various target platforms, such as browsers or mobile devices. The main idea behind this library is to simplify the development process by breaking down complex user interface elements into composable *React components*. Components are the main development units of React applications, and the library provides functionality to implement more complex per-component behavior, such as component-level storage or custom behavior of the UI elements. Components can be implemented as *class-based* or *functional* using *JSX*, which extends the standard JavaScript syntax.

Program 1 Comparison of JSX and Feliz syntax

```
//Component implemented using JSX
function SimpleComponent() {
  return (
      <div>
          <p>JavaScript component</p>
      </div>
  );
}

//Component implemented in F# using the Feliz DSL
let simpleComponent =
  Html.div [
      Html.p "Feliz component"
  ]
```

The *Feliz* [22] library provides a DSL to create functional React components in F# Fable projects. It combines the benefits of React, such as the ability to create composable UI elements, and the benefits of F#, such as type safety and functional programming capabilities. We can see an example of a component implemented in both JavaScript and React in Program 1.

2 Design principles and Requirements

This thesis aims to bring together the benefits of existing systems, such as the usability demonstrated by *Hypercard*, the responsive development loop of *Darklang* [6], and the ability to generate textual representation of programs based on interactions with a low-code programming interface, as demonstrated by *Sketch-and-Sketch*[7].

Unlike many commercially developed low-code programming systems that focus mainly on supporting specific usage scenarios, we want to approach the design by identifying design principles and following them as closely as possible during our implementation. The main design principles are the following:

1. **Data-driven approach**
2. **Low-code editing**
3. **Code generation capability**

Our work is then an exploration of a particular point in the design space of low-code programming systems. We want to show what a minimal, but cleanly designed system satisfying those principles would look like.

2.1 Data-driven approach

The main general design principle of the proposed programming system is the *data-driven* approach in context of creating web application UI elements. We define the general approach as follows:

Definition 2 (Data-driven approach)**.** *A software development approach where the creation of a particular software artifact is dependant on* ***concrete data*** *provided to the system.*

Meaning that the development process is tied to the specific data provided. *Darklang* [6] partly satisfies this definition, although it was not explicitly designed to be data-driven In the context of creating web application UI elements, some data can be used only for certain UI elements as to maintain the correct semantic meaning and functionality of the elements. Another definition which could be used to describe our approach is the following:

Definition 3 (Data to UI)**.** *Defined by Sahay et al. [4] as "A data-driven approach that starts from data modeling and then builds the user interface of the application followed by the specification of business logic rules and workflows."*

This definition also includes the concepts of data modeling and specification of business logic rules and workflows but we only focus on the aspect of user interface creation in this thesis. The main difference between our programming system and other systems described by Sahay et al. [4] as *Data to UI* such as *Mendix* [5] is that creating the user interface is directly tied to the data provided to the application. Usually, the *Data to UI* systems provide options to create pre-made UI elements, which

may then reference and use the provided data. Whereas our proposed system creates the UI elements based on the data itself which then reference the provided data.

The system analyzes the provided data and provides development options based on its structure and properties. The analysis can be tailored to the specific aims of the programming system. For example, the proposed application for creating web applications can analyze the data and map it to specific HTML elements based on its type and structure. Usually, the developer performs this data analysis and creates the GUI elements by hand. Using this approach, the developer can offload some of this work onto the system and focus on other aspects of the development process.

The data analysis can employ various techniques, such as data structure and data type recognition. One option is that the system performs the analysis once after the developer provides data to the system, and the system provides options based on this single analysis. Alternatively, the system can perform it continuously throughout the development process and provide development options based on the already created application elements, similar to Output-directed programming described in Section 1.1.3. The system could also use various machine-learning techniques to improve the data analysis.

Another valuable property of systems employing this approach is their ability to be used in conjunction with other existing technologies, mainly those focused on data modeling and retrieval. For example, the developer can use the proposed system employing this approach together with existing technologies such as *GraphQL* [29]. Using GraphQL, the developer can specify the data they wish to use and its structure. This data can then be provided to the system and used to create the desired web application.

2.2 Low-code editor

The second design principle is the concept of a *low-code editor*. Low-code editors usually provide a graphical user interface through which the developer interacts with the system and develops the desired application. The system provides contextual menus to aid in the creation of the application. Various existing low-code programming editors were presented previously in Section 1.1.

The visual design and functionality of the system is inspired by the ”what you see is what you get” (abbreviated as *WYSIWYG*) editors presented by Yang et al. [30]. This sub-category of low-code editors displays the application’s output as if running inside its native environment. This design allows the system to update the output every time a change is made to the program and display its result, allowing for a more dynamic development loop [8].

Our prototype system provides contextual menus based on the structure of the input data, as we will see in Chapter 4 and Chapter 5. The design of the contextual menus was influenced by menus in currently existing programming systems, such as property menus from the *Visual studio* editor.

2.3 Code generation

The last design principle is the ability to generate a plain–text representation of the created application. This ability addresses some of the concerns and main pitfalls

of many low-code programming systems, as stated by Pinho et al. [3]. These concerns include *Interoperability issues* and *Vendor lock-in*. Both are mitigated as the user can download the already created part of the application in its textual representation and continue its modification in another programming system.

Our prototype implementation also provides a code generation capability. The type system of the application represents the created parts of the application in a form that allows translation of the created application to various programming languages and other technologies. The prototype system can generate the resulting application in *HTML* and *plain JavaScript*, but could be extended to provide a wider selection of target technologies, such as *React* or *VueJS* frameworks.

2.4 Requirements

2.4.1 Functional Requirements

2.4.2 Non-functional Requirements

3 Core Logic - Domain and Operations

This chapter describes the core logic of the programming system. Firstly, we describe the hole-based approach applied to the creation of elements. After that, we describe the type system, mainly the internal representation of the UI elements and necessary operations on these types. We describe mapping the input data types to specific types representing the UI elements. Lastly, we describe how the system combines the input data and the corresponding UI elements. All example Programs shown in this Chapter are written in the F# programming language.

3.1 Hole-based Approach

The primary motivation of this approach is to allow *incremental creation* of UI elements. The elements are created based on the provided data and its structure. Each element corresponds to a data value. We need the ability to represent data values that have yet to be used to create UI elements. To address this need, we define a type named *Hole* as follows:

Definition 4 (Hole type definition)**.** *A Hole is a placeholder type representing a UI element that is yet to be created despite its existing corresponding data. It serves as a temporary stand-in during the incremental construction of a user interface.*

We then define the *incremental creation process* as a *sequence of opreations*. Each operation is either a *modification* of an existing UI element or a *replacement* of a Hole element with a new UI element based on the corresponding data. This approach allows the system to perform different tasks after each operation. These tasks include updating the UI element preview, analyzing the input data, or providing new modification menus and options. The system can also analyze the existing UI elements and perform other operations at this time.

3.2 Domain model and Mapping

In this section, we describe the types which the system uses to represent the input data and types used to represent the created UI elements. We also define the mapping between the input data and the UI element representation.

3.2.1 JSON Type

The creation process starts by providing *JSON* data to the system. In order to use this data, we parse it and create an internal representation. We define a discriminated union type named *JSON*. This type is the foundation for the Abstract Syntax Tree representation of the JSON data. We can see the type definition in Program 2.

Each node represents a JSON value in the input data. The types of nodes can be divided into two categories:

- **Collections:** The first category contains types representing a *collection* of other values. This category includes the types *JObject* and *JArray*. We define the two collection types as follows:
  - **JObject:** It is based on the JSON Object type and represents a collection of different JSON types. The original ordering of the inner elements is *ignored*.
  - **JArray:** It is based on the JSON List type and represents a collection of JSON values of the *same type and structure*. The original ordering of the inner elements is *preserved*.
- **Primitives:** The second category contains types representing data primitives such as numerical values, boolean values, a string of text, or the null value.

Program 2 JSON type

```
type Json =
  | JNumber
  | JString
  | JBool
  | JNull
  | JArray
  | JObject
```

3.2.2 RenderingCode Type

To facilitate the creation of UI elements based on the JSON type, we define a type named *RenderingCode*. The *RenderingCode* is a discriminated union type used to represent the UI elements. Similarly to the JSON type, each case represents a type of an HTML element with a corresponding mapping to the JSON type. This type is the foundation for the *Abstract Syntax Tree* representation of UI elements. We can see the type definition in Program 3.

Program 3 RenderingCode type

```
type RenderingCode =
  | HtmlElement
  | HtmlList
  | HtmlObject
  | Hole
```

We define each case type and what it consists of as follows:

- **HtmlElement:** Represents a single HTML element. It consists of:
  - *Tag*: Denotes the HTML tag (e.g., div, p, span).
  - *InnerValue*: Represents the content of the element defined as seen in Program 4. Where *Data* indicates dynamic content from input JSON, *Empty* represents an empty element, and *Constant* holds static, developer-defined content.

- *Attributes*: A collection of key-value pairs representing HTML attributes. It consists of a string key and an InnerValue value. This representation allows for the selection of an attribute's value from the input data.

Program 4 InnerValue type definition

```
type InnerValue =
  | Data
  | Empty
  | Constant
```

- **HtmlList:** Represents a collection of HTML elements of the same type corresponding to JSON arrays. It includes:
  - *ListType*: Specifies the list type (e.g., unordered, ordered, table).
  - *itemCodes*: A list of RenderingCode elements representing list items.
- **HtmlObject:** Represents a collection of diverse HTML elements derived from JSON object. It consists of:
  - *ObjectType*: Defines the type of the object (e.g., div, span, Section).
  - *Elements*: A list of RenderingCode elements representing the object's contents.
- **Hole:** Represents a placeholder in the UI element structure as defined in Section 3.1, allowing for dynamic element creation.

Each RenderingCode type, with the exception of the type *Hole*, also provides the ability to add custom event handling. Each elements includes a list of user-defined event handlers. Each event handler consists of the name of the event, and a script that is executed. This script is user-defined and implemented in JavaScript.

3.2.3 Data Mapping: JSON to RenderingCode

Transforming JSON data into UI elements requires systematic mapping between JSON and RenderingCode types. This mapping forms the core of our UI generation system, allowing us to convert arbitrary JSON input into a structured representation of UI elements. Each JSON type has a corresponding representation in RenderingCode:

- JObject maps to HtmlObject
- JArray maps to HtmlList
- JNull, JString, JNumber, and JBool map to HtmlElement

The *incremental creation process* described in Section 3.1 involves dynamically creating replaceable *Hole* elements. We perform this creation lazyly and incorporate it into the mapping process. A *Hole* element is created for each inner element of the collection types described in Section 3.2.1.

We call this mapping process the *recognition* of the JSON value and define the mapping function named *recognizeJson* in Program 5.

Program 5 JSON to RenderingCode mapping

```
let recognizeJson (json) =
    match json with
    | JArray array ->
        //create a hole for each element of the array
        HtmlList containing the holes as elements
    | JObject obj ->
        //create a Hole for each element of the object
        HtmlObject containing the holes as elements
    | JNull -> Hole(UnNamed)
    | _ -> HtmlElement
```

3.3 Incremental Creation Process

We described the *incremental creation process* briefly in Section 3.1 as a sequence of discrete operations. In this Section, we extend this description and define the previously described operations in greater detail. We divide the operations into two categories:

- **User operations:** Operations performed by the user such as element creation, element modification, and providing data to the system. User operations are performed through the provided GUI.
- **System operations:** Operations performed by the system in reaction to the User operations. These operations include creating new RenderingCode elements, analyzing newly visited JSON values, modifying AST, rendering menus, and previewing elements.

This categorization allows us to divide the functionality and responsibilities of the system between different parts of the application. For example, the GUI portion of the application is mainly responsible for accepting User operations, whereas other modules can implement different System operations.

3.3.1 Traversal

An essential aspect of the creation process is how the system processes the JSON AST and helps the user build the RenderingCode AST. The process involves traversing both the JSON and RenderingCode ASTs simultaneously. We can do this because we create the RenderingCode AST based on the structure of the JSON AST. This allows the system to dynamically create Hole elements for direct descendants of existing RenderingCode elements for which corresponding data exists, but the elements have not yet been created.

Another way to look at this process is that the RenderingCode AST is guiding our traversal, while the JSON AST inspires the structure of the RenderingCode AST. The traversal begins from the roots of the ASTs, and we traverse the trees recursively. Based on the type of the visited RenderingCode node and the mapping described in Section 3.2.3, we can perform different operations:

- **HtmlElement:** The node is of type HtmlElement, which maps to a primitive JSON value. This means the JSON value has no descendants, and we can end

the traversal. The system then performs the operations of displaying a preview of the HtmlElement and a modification menu.

- **Hole:** The node is of type Hole, meaning the JSON value was visited before, but the user has not created the corresponding RenderingCode. The presence of a Hole element also means that we cannot visit the potential descendants, and we can stop the traversal. The system displays a menu to add a new element corresponding to the JSON value.

- **HtmlList and HtmlObject:** When the node represents a *Collection* type of type HtmlList or HtmlObject, we recursively traverse all its descendants. The system displays a modification menu for the element and continues the traversal.

After every User operation, the system performs this traversal to update the element preview and modification menus to reflect changes made to the RenderingCode AST.

3.3.2 Hole Replacement and Modification

User operations require finding a specific node inside the RenderingCode AST during the creation process. We define a dynamically generated *Path* for all nodes during the traversal process to find this specific node. The *Path* is a sequence of indices unique to every node in the AST.

Program 6 Example: RenderingCode AST with corresponding paths

```
HtmlObject [ //Path: []
    HtmlElement(Div, [], Empty) //Path: [0]
    HtmlList(UnorderedList, [ //Path: [1]
        HtmlElement(Li, [], Constant "Item 1") //Path: [1,0]
        HtmlElement(Li, [], Constant "Item 2") //Path: [1,1]
      ])
  ]
```

We can see the different paths for each element in Program 6, which shows an example of RenderingCode AST. The example AST consists of a root node of a type HtmlObject and elements contained within it. We see that the root node has an empty path. Then, we append the position of each element inside the collection to the Path. This allows us to traverse the AST based on the specified Path and find the correct element using a recursive search function.

Using the dynamically created paths, we can replace a RenderingCode by following its specified Path through the RenderingCode AST. We define a recursive function named *replace* which navigates the AST using the provided path to locate and replace a specific element with a new one. We can see a description of the potential implementation in Program 7.

The User operations such as replacing a Hole with a new RenderingCode or modification of an existing RenderingCode correspond to a specific usage of the replace function.

Program 7 A function used to replace a RenderingCode inside the RenderingCode AST

```
let rec replace
  (path: int list)
  (replacementCode: RenderingCode)
  (currentCode: RenderingCode) =
  match path with
  | [] -> replacementElement
  | head :: tail ->
      match currentCode with
      | HtmlList ->
          // recursively call the function on a collection element
             with the index equal to head
          // return new HtmlList
      | HtmlObject ->
          // recursively call the function on an element with
          // the key located inside the keys collection, that has
             index equal to head
          // return new HtmlObject
      | _ -> currentCode // remaining path, but the current element
         is not a collection
```

3.3.3 Creation of a single RenderingCode

Now that all necessary operations are defined, we focus on the creation process of a single RenderingCode element based on the input data. Creating a single RenderingCode element consists of multiple System operations and a single User operation. We can see the sequence of operations in Figure 3.1.

The process starts when the system visits a previously unvisited node of the JSON AST during the traversal described in Section 3.3.1. The system automatically creates a corresponding Hole element and adds it to the RenderingCode AST. Then, the system displays a menu to the user, allowing them to replace the Hole element with a RenderingCode based on the JSON value. After the user chooses to add the new RenderingCode element, the system uses the previously defined *recognizeJson* function to create the new element based on the corresponding JSON value. Following that, the system uses the *replace* function to replace the existing Hole element at the specified path with the newly created RenderingCode. Lastly, the system traverses the modified RenderingCode AST and JSON AST as described in Section 3.3.1 and performs operations such as displaying modification menus and element preview.

3.3.4 Combining RenderingCode and JSON data

Once the RenderingCode AST is created, the system needs to combine the RenderingCode elements with the data from the JSON AST. We use *Structural referencing* to combine a RenderingCode element with the corresponding JSON value. By Structural referencing we describe a process where a RenderingCode with a specific assigned Path is combined with a JSON value with the same Path. We can do this easily, as the structure of the RenderingCode AST mirrors that of the JSON AST, with the exception of CustomWrappers and CustomElements, which are ignored during the process and we preserve the paths of wrapped RenderingCode elements. Thanks to the approach of Structural referencing, we can perform this process during our previously described

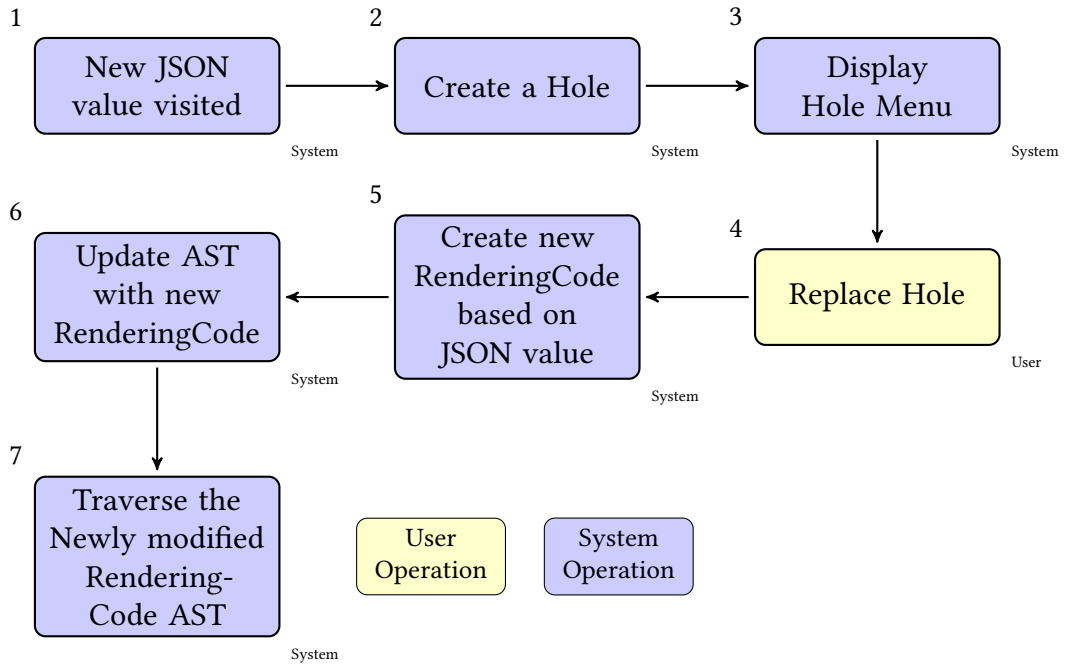

Figure 3.1 Single RenderingCode Creation Process

traversal in Section 3.3.1.

3.4 Summary

In this Chapter, we described the the core logic and domain of our programming system. This core logic serves as the foundation of the prototype programming system, which we will describe in the following Chapter.

- In Section 3.1, we defined the Hole-based approach to creating UI elements based on concrete data.
- In Section 3.2, we defined the Model of the application consisting of types representing the provided data and UI elements. We also described the mapping between the input data and our internal representation of created UI elements.
- In Section 3.3, we described the *Incremental Creation Process* including the User and System operation. Then, we described the simultanious traversal of the JSON AST and RenderingCode AST. After that, we described the process of replacing and modifying the RenderingCode AST using a Path-based approach. Lastly, we described the process of combining the RenderingCode elements with correspond JSON values.

4 Implementation

We will describe the implementation according to the design principles described in Chapter 2 and the core logic described in Chapter 3. We will explore the technologies chosen, the system's architecture, and key implementation details of *Data-driven UI* programming system.

4.1 Technologies

To provide context for the implementation, we'll first examine the technologies used in developing the *Data-driven UI* system. We decided to create the *Data-driven UI* editor in a from of a browser-based client-side application providing a graphical user interface, rather than a traditional desktop application. We made this decision to allow greater cross-platform compatibility and the ability to see the preview of the created web applications in a browser-based environment. We also wanted to implement the system using a programming language with functional programming capabilities and a strong ecosystem, which narrowed our range of suitable options. The system is implemented using the following key technologies:

- **F#:** The *F#* [20] programming language is used to implement the entire application, including the core logic and the user interface, chosen for its strong type system and functional programming capabilities.
- **Fable:** The *Fable* [21] compiler, briefly described in Section 1.2.4, compiles the F# source code to JavaScript, enabling browser-based execution and using technologies from the JavaScript ecosystem.
- **React:** The *Feliz* [22] library provides a domain-specific language (DSL) for building *React* user interface components and applications in F#.
- **Elmish:** *Elmish* [10] is a library used to enable the creation of Elmish style applications in F#, which follow the MVU pattern described in Section 1.2.3.
- **Tailwind:** We use the Tailwind CSS framework for the layout and styling of the UI components of the application, which provides composable CSS classes and enables high customizability of the UI elements.
- **SimpleJson:** The *Fable.SimpleJson* [24] library is used to parse the input JSON data into the internal representation described in Section 3.2.1.
- **SAFE stack template:** We use the *SAFE stack* [31] template's *Build* project, which provides scripts for building web applications built in F#.

4.1.1 Alternatives

While we selected the technologies mentioned in the previous Section for our implementation, for several of them we considered using alternative technologies:

- **Programming Language:** Instead of F#, we could have used other functional programming languages such as Haskell or OCaml. These languages also offer strong type systems and functional programming capabilities. However, F# was chosen for its seamless integration with the .NET ecosystem, high-quality development tools and documentation, and its ability to be compiled to JavaScript via Fable.

- **CSS Framework:** We considered using alternative CSS frameworks, such as Bulma or Bootstrap, which provide pre-made styled components. We chose to use Tailwind instead, as the composable styling classes allow for a more direct approach to styling UI elements, instead of trying to adapt and modify the pre-made components.

- **JSON Parsing:** Instead of Fable.SimpleJson, we could have used the closest alternative library called *Thoth.Json* [32]. The main strenght of this library is the ability to create custom JSON *encoders* and *decoders*. However, as we need the ability to parse JSON data of abitrary strucuture, SimpleJson's lightweight nature and its internal representation of the parsed data made it our preffered choice.

While these alternatives have their strengths, our chosen technologies provided the best balance of functional programming capabilities, browser compatibility, and ecosystem support for our specific requirements.

4.2 System architecture

Before we describe the implementation specifics, we must first describe the *architecture* of the implementation. The implementation is divided between different F# *modules*, each providing different functionality. We define two main modules named *Core Logic* and *Editor*, which contain sub-modules implementing the specific functionality. These modules can be described as follows:

1. **Core Logic module:** Contains modules responsible for implementing the Core logic described in Chapter 3.

2. **Editor module:** Modules focused on implementing the user interface of the programming system and its functionality following the Elmish architecture described in Section 1.2.3. They use and depend on the functionality provided by the CoreLogic module.

We will describe the main modules in greater detail in the following Sections.

Each sub-module comprises various functions or type definitions. In our implementation, we separate type definitions and functions into distinct modules, similarly to the separation of *Domain* and *Infrastructure* layers of the *Domain-driven* architecture. The modules containing the type definitions define the system's domain and have either none or a small number of external dependencies. The modules containing the functions implement the behaviors and operations that manipulate and utilize the domain types.

This approach inherently makes the application more easily extensible. For example, we can add new functionality to the Editor module without the need to change the implementation of the Core Logic sub-modules.

4.2.1 Core Logic Module

The Core Logic module is the first main module of the *Data-driven UI's* implementation. It forms the foundation of our *Data-driven UI* system, implementing the types and operations described in detail in Chapter 3. Figure 4.1 illustrates the overall structure of this module. We divide the implementation into the following sub-modules:

- **Types:** This module comprises sub-modules that define the fundamental data structures of our system:
  - **RenderingTypes:** Contains definitions for types such as *RenderingCode*, *CustomWrapper*, and other related types.
- **Operations:** This module includes sub-modules that implement various operations on the types defined in the Types module:
  - **RenderingCode:** Implements crucial operations such as the *replace* function, which modifies the RenderingCode AST. This module is central to manipulating our UI representation.
  - **DataRecognition:** Handles the mapping process between input data and our internal UI element representation. It includes the *recognizeJson* function, detailed in Section 3.2.3, which is key to our data-driven approach.
  - **CodeGeneration:** Provides functionality to generate textual representations of created web applications from RenderingCode ASTs. This module bridges our internal representations with output code.

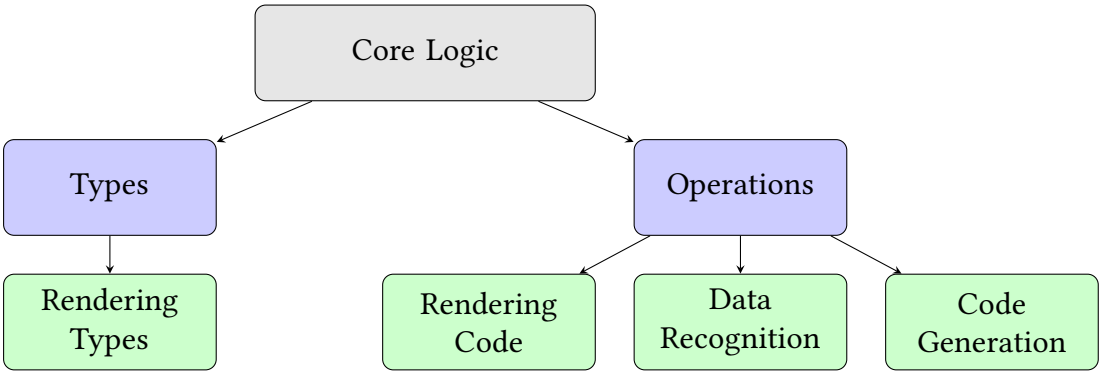

Figure 4.1 Core Logic Module Structure

This modular structure allows for clear separation of concerns, with types and operations distinctly categorized. It improves the extensibility and maintainablity of the Core Logic functionality.

4.2.2 Editor module

The Editor module is the second main module of the *Data-driven UI's* implementation. It is responsible for implementing the programming system's UI and functionality. It facilitates tools from the Core Logic module and technologies described in Section 4.1

such as Elmish, to provide an interactive low-code editor environment. Figure 4.2 illustrates the structure of the module.

We divide the module into the following sub-modules:

- **Types:** This module defines the core data structures that represent the state and operations of our Elmish-style application, as detailed in Section 1.2.3.
  - **EditorDomain:** Encapsulates the domain types that represent the internal state of the Data-driven UI application.
  - **PageEditorDomain:** Defines specialized domain types focused on representing the internal state of the Page Editor component.
- **Utilities:** Provides a collection of essential helper functions that support various aspects of the application:
  - Icon importing and management.
  - File upload handling and processing.
  - JSON data parsing and serialization, leveraging the *Fable.SimpleJson* [24] library.
  - Importing the libraries necessary to use the Codemirror editor component.
- **Components:** Implements a suite of custom React components that form the interactive user interface:
  - **CustomRendering:** Implements dynamically rendering previews of the RenderingCode AST, coupled with interactive modification menus that adapt to the current editing context.
  - **OptionComponents:** Provides components serving as context menus to allow editing functionality for the individual RenderingCode elements.
  - **EditorComponents:** Provides a set of versatile components that deliver general editor functionality, including a tab management system inspired by industry-standard editors like Visual Studio Code.
  - **PageEditorComponents:** Offers specialized components designed for creation, real-time preview, and efficient modification of the RenderingCode AST.

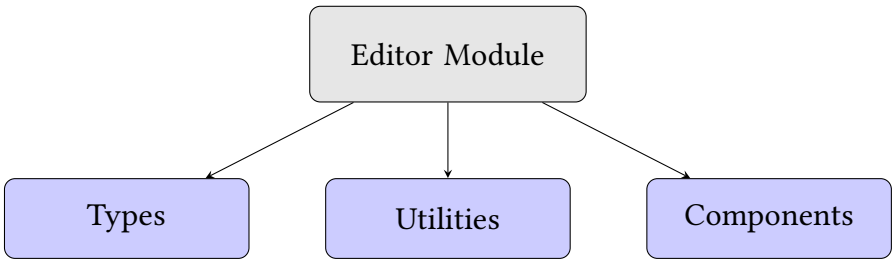

Figure 4.2 Editor Module Structure

4.3 Main Features

Now that we have defined the implementation's modular architecture and structure, we can describe the specifics of the implementation. As the implementation comprises many different type definitions and functions of various complexity, we will briefly describe the system's *main features* that implement the design principles described in Chapter 2.

4.3.1 User interface implementation

The integral component of our programming system is the low-code user interface, as it realizes one of the design principles described in Chapter 2. It provides tools for creation and modification of UI elements through the use of context menus. It also shows the preview of the already created UI elements.

To implement the programming system, we use tools and technologies previously described in Section 4.1. One of these tools is the *Feliz* [22] library, used for creating composable React UI components in F#. Another important tool is the *Elmish* [10] library, which provides abstractions that enable the creation of *Elmish-style* applications in F#. We can also create React components that use the Elmish-style architecture as part of their internal representation of state and operations by using the *React.useElmish* hook provided by the *Feliz.UseElmish* library.

Application-Level Elm Architecture

At the top level, our application follows the Elm architecture described in Section 1.2.3. The primary application state is represented by the *Model* type, and we define corresponding *View* and *Update* functions to manage this state. We can see the *Model* type representing the entire application's state in Program 8.

Program 8 The Model type representing the state of the *Data-driven UI* system.

```
type Model = {
    Pages: Map<Guid, Page>
    OpenTabs: Tab list
    ActiveTabId: Guid option
    IsSidebarOpen: bool
}
```

The representation shows that we provide tabbing functionality similar to existing programming systems, such as *Visual studio code*. This means the user can create and edit multiple Pages simultaneously, switching between them as needed.

Component-Level Elmish Architecture

For specific components that require more complex, localized state management, we utilize the previously described Feliz.UseElmish library. This approach allows us to create Elmish-style React components that have their own local state, update logic, and view functions while still being part of the greater Elm-style application.

For example, the *PageEditor* components use the *Feliz.UseElmish* library to manage their internal state. This structure allows the PageEditor to maintain its own state

and update logic, making it more modular and accessible to reason about while still communicating with the main application when necessary through the use of Elmish *Commands*. We can see the *PageEditorModel* representing the state of the PageEditor React component in Program 9.

Program 9 The PageEditorModel type representing the state a PageEditor component.

```
type PageEditorModel = {
    PageData: Page
    FileUploadError: bool
    IsPreview: bool
}
```

4.3.2 User Interface layout

The user interface is composed of multiple React components some managed by the top-level Elmish application, while others employing the component-level architecture. We can see the layout of the *Data-driven UI's* interface in Figure 4.3

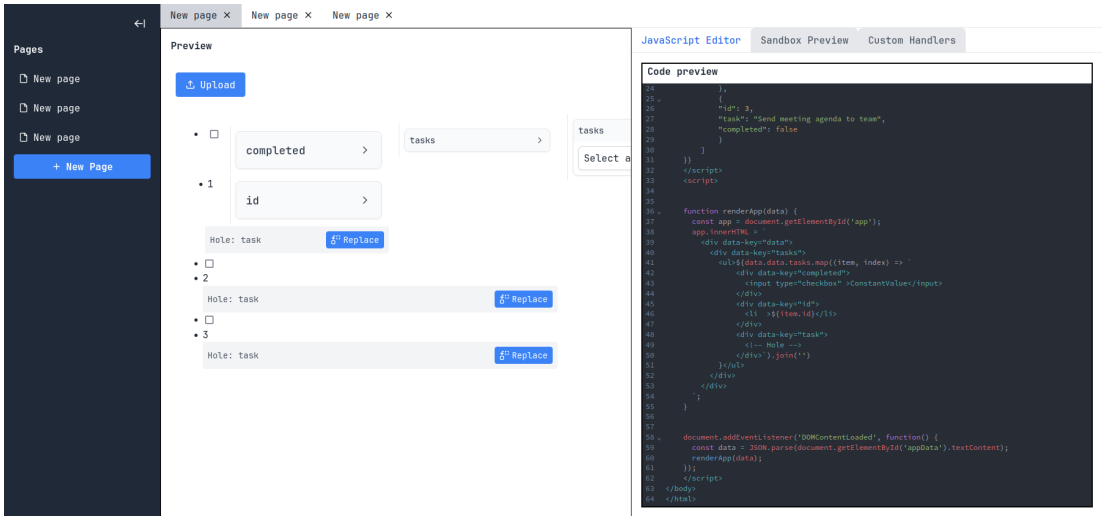

Figure 4.3 Example of *Data-driven UI's* user interface

Located on the left is a collapsable side-menu used to open created pages for editing. On the top we can see a panel displaying tabs for currently open pages. The state and operations of these components is managed by the main Elmish application.

Under the tab panel we can see the main editor window. The left side of the window is used to provide the modification and preview functionality explained in more detail in the following Section. On the right side we see a window for creating and modifying custom JavaScript handler functions. The state and operations of these components is managed by the individual PageEditor components.

4.3.3 Custom Rendering

To implement the Low-code approach described in Chapter 2 combined with the Data-driven approach described in Chapter 3, we define a *Higher-order* recursive func-

tion named *renderingCodeToReactElement*. This function performs the simultaneous traversal of the RenderingCode and JSON ASTs described in Chapter 3.

The arguments of the *renderingCodeToReactElement* function are the following:

- *code*: The RenderingCode to be rendered
- *path*: The current path in the RenderingCode AST
- *json*: The corresponding JSON data
- *name*: A string identifier for the current element corresponding to a JSON field of the same path
- *dispatch*: A function to dispatch messages to the Elmish update function

The function recursively traverses the RenderingCode AST, matching each type of RenderingCode to its appropriate rendering function. We then define a rendering function for each type of RenderingCode. Each rendering function combines the RenderingCode structure with the corresponding JSON data to create a preview of the element. The function also interweaves option menus with the rendered elements when, allowing for editing of the rendered structure. Error handling is implemented throughout the rendering process to handle mismatches between the RenderingCode and JSON structures.

This function provides the main functionality of our implementation and effectively combines the functionality from the Core logic module, with the low-code approach.

4.3.4 Code generation

One of our core design principles, as outlined in Chapter 2, is the ability to generate a textual representation of the created application.

To implement this functionality, we define a recursive function named *generateHtml*. This function operates in a manner similar to the *renderingCodeToReactElement* function described in the previous section, performing a simultaneous traversal of the RenderingCode and JSON ASTs as detailed in Chapter 3.

For each type of RenderingCode, the function generates appropriate HTML:

- `HtmlElement` is translated into its corresponding HTML element
- `HtmlList` generates HTML lists based on the list type
- `HtmlObject` creates a structured representation of the object based on its properties and object type
- `Hole` elements are represented as comments in the generated HTML, maintaining the structure for potential future updates

A key feature of this function is its ability to automatically populate the generated HTML elements with the corresponding JSON data, ensuring that the textual representation accurately reflects both the structure and content of the created application.

The generated HTML is formatted with proper indentation for readability, making it easier for developers to understand and potentially modify the code outside of the low-code environment. This approach not only facilitates the transition from low-code to traditional development when necessary but also promotes transparency in the development process.

4.4 Building and Deployment

Now that we have described the implementation of the *Data-driven UI* system, we must describe how the application is built and deployed. As we mentioned in Section 4.1, we use the *Build* project provided by the *SAFE stack* [31] template. The original SAFE stack *Build* project supports building *full-stack* applications written in F#, and we modified it to remove the support for building the *Shared* and *Server* projects, as our programming system only consists of a client-side application.

4.4.1 Build Tools and Libraries

The SAFE stack *Build* project eploys different specialized tools to make building F# application easier. The two main tools used by the *Build* project are the following:

- **FAKE (F# Make):** A build automation tool for projects written in F#, used to define build targets and their individual build steps.
- **Vite:** A web development tool for building JavaScript client-side applications, which provides a development server environment with Hot Module Replacement capability.

The pre-requisites are needed to build the application:

- **.NET Core SDK**
- **Node 20**

4.4.2 Build targets

The *Build* project provides multiple different build targets. Each provides a different build result and comprises a different sequence of build steps.

The *Build* project defines the follwing targets:

- **Run:** Configuration for running a local development server with HMR capabilities. Uses the Fable compiler to compile the source code into JavaScript and uses Vite to run the development server.
- **Bundle:** Target used for bundling of the application for deployment. Uses the Fable compiler to compile the source code into JavaScript and uses Vite to bundle and optimize the application assets, including the JavaScript program, CSS, and static files.
- **RunTests:** Configuration for running the unit tests.
- **Format:** Uses the *Fantomas* F# code formatter, to format the source code.

In order to build a specific target, we use the following command:

```
dotnet run [Target]
```

It consists of the *dotnet run* command, followed by the name of the target we wish to build. It is important to mention, that the target names are case-sensitive.

4.4.3 Deployment

We provide a deployment solution in a form of a `Dockerfile`. To run or deploy the *Data-driven UI* docker image, we must first build it using the following command inside the root of the project:

```
docker build -t data-driven-ui .
```

After the build process is finished, we can run the docker container by running the following command:

```
docker run -p 8080:80 data-driven-ui
```

4.5 Testing

To decide if the implemented operations perform as expected, we decided to create a separate project named *Tests*. We implemented *Unit tests* for various operations from the Core Logic module, such as the *replace* function from the *CoreLogic.Operations.RenderingCode* module, and the code generation capability from the *CoreLogic.Operations.CodeGeneration* module.

We use the testing library named *Fable.Mocha* [33], which provides useful tools to implement the testing functionality. It provides a browser-based testing environment with a graphical user interface displaying the individual test cases and their results. It also allows composing related *test cases* into *test lists*, improving the readability and maintainablity of the tests.

The structure of the individual test cases follows the *Arange Act Assert* (AAA) unit test pattern. It involves preparing the required data, then performing some operation using this data, and then asserting that the operation has the expected result. We can see an example test case in Program 10. We can see the name of the test case and its implementation which follows the *AAA* pattern.

Program 10 An example test case testing the functionality of the *replace* function.

```
testCase "Replace Hole"
  <| fun _ ->
      let original = Hole(Named "placeholder")
      let replacement = HtmlElement(Div, [], Empty, [])
      let result = replace [] replacement original
      Expect.equal result replacement "Should replace Hole at root
         level"
```

4.6 Summary

In this chapter, we described the practical implementation of our Data-driven UI programming system, translating the theoretical concepts and core logic into a functional prototype. The key points covered in this chapter are:

- In Section 4.1, we outlined the technologies used in our implementation, including F#, Fable, React, Elmish, and others. We also discussed alternative technologies considered and our rationale for our choices.

- In Section 4.2, we presented the system architecture, detailing the modular structure of our implementation. We described the Core Logic and Editor modules, explaining how they interact and their respective responsibilities.
- In Section 4.3, we explored the main features of our system, focusing on:
  - The user interface, which combines an application-level Elm architecture with component-level Elmish architecture for more complex elements.
  - The custom rendering system, which implements the low-code and data-driven approaches through the `renderingCodeToReactElement` function.
  - The code generation capability, which translates our internal representations into HTML and JavaScript.
- In Section 4.4, we described the build and deployment process, detailing the tools used and the available build targets.
- In Section 4.5, we outlined our testing approach, explaining how we use unit tests to verify the correctness of our core operations.

5 Benchmarks

This chapter evaluates our Data-driven UI prototype system on two tasks from the *7GUIs benchmark* [12]. Our primary goal is to assess whether we can implement these tasks according to their specifications by using only our prototype system and to determine if our approach successfully reduces the amount of code that needs to be written, as per the definition of low-code programming systems by Pinho et al. [3].

5.1 Methodology

We will evaluate our prototype system based on the following criteria:

- **Successful implementation of all specified functionality:** We must implement the exact functionality described by each task. If we cannot implement a specific task as specified, we can reason about our system's limitations and potentially identify problems with our implementation or design.
- **Number of lines of code written:** We will compare the number of lines of code required in our system versus the provided referential solution. We will only consider *physical lines of code* as defined by Park [34]. Because our system requires concrete data before we can begin building the desired application, we will also include the Number of lines of code needed to be written for the data preparation. We will also ignore all lines of code related to styling the elements and focus only on implementing the specified functionality and UI structure.

For each successfully comleted task, we will:

1. Describe the task requirements
2. Outline the implementation process using our Data-driven UI system
3. Present the resulting application or describe what functionality cannot be implemented
4. Analyze the Number of lines of code written

5.2 To-do list application

5.3 Counter Task (7GUIs)

The Counter task is defined by the *7GUIs benchmark* [12] as follows: "The task is to build a frame containing a label or read-only textfield T and a button B. Initially, the value in T is "0" and each click of B increases the value in T by one."

5.3.1 Creation process

The steps of the creation process are the following:

1. **Prepare JSON data:** The creation process starts by creating JSON data based on the format specified by the task. We can see the created JSON object in Figure 11.

2. **Upload data to the system:** We upload the created JSON data to the system.

3. **Replace holes with new elements:** This step involves clicking on a provided button menu for each Hole element.

4. **Modify the elements using the context menus:** We use the provided context menus to change the tag of each element. We select the label tag for the label element and the button for the button element.

5. **Implementation of custom behavior:** We select the *Custom Handlers* menu. We type the name of our custom function into the provided input field and we name the function as `plusOne`. Then, using JavaScript, we implement the desired behavior of increasing the value of the label field by one every time this function is called. Into the editor window, we input the following: `data.label++;`. We do not need to write anything else, as the system automatically generates the function's body and other event handlers.

6. **Add the "onClick" handler to the button element:** We use the menu for the *button* element to add an event handler which calls the `plusOne` function when triggered.

Program 11 JSON object created as input for the Counter task (7GUIs)

```
{
  "label": 0,
  "button": "count"
}
```

5.3.2 Results

We *successfully* created the desired application, its specified functionality, and UI elements.

5.3.3 Analysis

The referential 7GUIs solution [35] for the counter task consists of 11 LOCs.

We will now analyze the Number of LOCs needed to implement the Counter task using our programming system

- In Step 1, we created the input data and wrote a total of 4 LOC.
- In Steps 2-4, we created and modified the UI elements using only mouse-based operations, resulting in 0 LOC written.
- In Step 5, we created a custom function, for which we wrote its name and implementation, resulting in 2 LOC written.

- In Step 6, we added a handler to the `button` element through mouse-based operations, resulting in 0 LOC written.

We wrote *six* lines of physical code in total, and the rest of the operations were performed through a mouse-based interface.

5.4 Temperature Converter Task (7GUIs)

Temperature Converter task by the *7GUIs benchmark* [12] involves creating a user interface with two input fields for Celsius and Fahrenheit temperatures, implementing bidirectional conversion between the two fields when valid numerical input is provided.

5.5 Evaluation

In this section, we summarize our findings from the benchmark tasks. Our evaluation process involved attempting to implement three distinct tasks using the *Data DrUId* prototype system.

6 Discussion

Every approach to software development has associated benefits and limitations. For example, Pinho et al. [3] identified several benefits and pitfalls associated with existing programming systems employing the Low-code programming approach. Some of the common benefits and limitations of existing Low-code programming systems are the following:

- Common Benefits:
  - *Low requirements for technical skills*
  - *High speed/short development time*
- Common Pitfalls:
  - *Interoperability issues*
  - *Vendor lock-in*

In this chapter, we will critically evaluate our work and describe its benefits, limitations, and potential avenues for future research. We will separate the limitations of the Data-driven Low-code approach from the limitations of our Data-driven UI programming system prototype as to better evaluate the potential feasibility of the approach itself.

6.1 Evaluation of the approach

This section evaluates the data-driven low-code approach and discusses its positive and negative aspects, separate from our *Data-driven UI* prototype implementation.

6.1.1 Benefits of the approach

Our work identifies several benefits of the Data-driven Low-code approach to creating client-side web applications. Each benefit directly results from our specific design principles described in Chapter 2. The main benefits are the following:

1. **Guided UI element creation:** The Data-driven Hole-based approach allows for a guided process of creating the desired UI elements. The input data inspires the created elements and their resulting structure. This process relieves the cognitive load associated with creating UI elements from scratch, as the system automatically creates the right type of element and places it at the correct place in the internal representation of UI elements.

2. **Mitigates Vendor lock-in:** The issue of Vendor lock-in usually stems from a system's opaque internal representation of the created software artifacts and the system not allowing the conversion of this internal representation to a more widely used representation. This means the user cannot easily use a different tool to modify or extend the created software artifacts.

   Our design mitigates this problem, as the ability to generate a textual representation of the created software artifact is a core requirement for systems implementing our approach, mitigating the possibility of Vendor lock-in issues.

3. **Mitigates Interoperability issues:** Another issue our approach mitigates is the lack of interoperability with other existing systems. Our system accepts input data in a widely used JSON format. The programming system can then be used alongside existing systems, mainly those focused on data manipulation and retrieval. An example of a programming system that could achieve a perfect synergy with systems implementing our approach is *GraphQL* [29].

   Another aspect of our approach that improves interoperability is the ability to generate a textual representation of the created program using a standard representation, which allows the use of other tools that accept the standard representation.

The described benefits may serve as a motivation to implement new systems employing our described approach or incorporate our approach into existing systems.

6.1.2 Limitations of the approach

Even though our approach provides several previously described benefits, we also identify several potential limitations. Some of these limitations are a direct result of our design principles described in Chapter 2, while others are a side-effect of our definition of the core logic described in Chapter 3. The potential limitations are the following:

1. **Necessity to prepare data in advance:** As defined in Chapter 2, the data-driven approach first involves providing concrete data to the system. By some users, this can be seen as a potential negative, as creation of a simple page mandates creating the structured contents of it first. This high upfront cost could be lowered by providing simple data templates which the user could modify according to their needs.

2. **Rigid element structure:** The creation process described in Chapter 3 defines a structured way of creating elements based on concrete data. We also define the concept of *structual referencing*, which enables us to dynamically create *Holes* and efficiently combine the created UI elements with corresponding data. However, we must keep the representations' structures exactly the same to ensure the correct structural mapping. This prevents us from modifying the structure of the created UI elements independently from the corresponding data, resulting in a very rigid structure.

   To mitigate this issue, we could explore other options for referencing the data from the UI elements, such as *Selector-based* referencing. This approach involves creating UI elements independently of the input data's structure and referencing it directly. However, we did not explore this option because we wanted to evaluate the purely data-driven approach.

3. **Potentially time and resource intensive:** The incremental creation process described in Chapter 3 involves simultaneously traversing the Abstract Syntax Trees representing the created UI elements and the input data. We must visit each node of the ASTs during our traversal to provide correct modification menus and previews for every created UI element. This means the time to traverse the created ASTs grows linearly with the number of elements they

contain. This approach to updating the preview and modification menus could pose a problem when modifying large ASTs, as we perform the traversal after each update of the UI elements. This means that the delay between changing a UI element and seeing the updated result could become too large.

This problem can be mitigated by employing a more targeted approach to re-rendering, such as re-rendering only the modified element and its children.

6.2 Evaluation of the implementation

In Chapter 4, we described the implementation of our *Data-driven UI* programming system. The motivation for creating this prototype system was to show a minimal but cleanly designed system satisfying design principles defined in Chapter 2. In this section, we evaluate the *Data-driven UI* prototype and discuss its positive and negative aspects in implementing the functionality of the data-driven low-code approach.

6.2.1 Positive aspects of the implementation

During our benchmarking process described in Chapter 5, we identified several positive aspects of the *Data-driven UI* prototype system. These strengths demonstrate the potential of our data-driven low-code approach and provide a foundation for future development. The key positive aspects are as follows:

1. **High-speed UI element creation and modification:** The system facilitates rapid creation of UI elements based on the structure of provided concrete data. For each created element, the system offers a modification menu enabling quick property changes through mouse-based or keyboard-based operations. Properties update automatically after each modification, allowing a fast and interactive development loop.

2. **Live preview with dynamic updates:** The system provides a real-time preview of created elements, automatically populated with corresponding data. This preview updates instantly with each modification, offering immediate visual feedback. Additionally, a sandboxed preview of the entire web application allows users to test and refine custom behaviors.

3. **Comprehensive code generation:** The system automatically generates a complete textual representation of the created web application. This includes incorporating user-defined custom behaviors and generating all necessary HTML elements, providing a seamless transition from the low-code environment to traditional code if needed.

6.2.2 Limitations of the implementation

While the Data-driven UI system demonstrates the potential of our approach, the prototype implementation has several limitations that should be addressed by future research:

1. **Handling of deeply nested data structures:** The current implementation struggles with efficiently displaying the preview and modification menus of

deeply nested data structures. This limitation can make it challenging for users to work with complex JSON inputs.

2. **Limited set of available UI components:** The current implementation supports a limited set of HTML elements and UI components. This restriction may not adequately cover all use cases for web application development, particularly for more complex or specialized applications.

3. **Performance with large datasets:** As noted in the approach limitations, the system's performance may degrade with very large JSON inputs due to the traversal method used. This implementation-specific issue could be mitigated with more optimized rendering and update algorithms.

4. **Lack of persistent storage:** Our prototype system does not implement any persistent storage functionality. This means that all data is kept in memory, and is lost upon refreshing the page.

These limitations primarily stem from the prototype nature of the implementation and the focus on demonstrating the core concepts of the data-driven low-code approach.

6.3 Future work

In this thesis, we laid the foundation for the data-driven low-code approach to creating client-side web applications. However, there are several areas where further research and development could significantly enhance the capabilities and applicability of this approach:

1. **Empirical user studies:** Conducting comprehensive user studies to evaluate the effectiveness of this approach compared to traditional development methods and other low-code platforms would provide valuable insights for further improvement.

2. **Solving the issue of rigid element structure:** Research focusing on the issue of strong coupling between the representations of UI elements and input data could enhance the overall functionality of the approach.

3. **Improved handling of complex data structures:** Research into more efficient algorithms for traversing and rendering large or deeply nested data structures could improve the system's performance and usability.

4. **Expanded UI component library:** Developing a more comprehensive set of UI components and patterns would increase the system's versatility.

5. **Modification of behavior through a low-code interface:** In our implementation, defining custom behavior of elements involves writing code in JavaScript. Future research could focus on defining and implementing comprehensive low-code interface for implementing the custom behavior without the need to write code.

6. **Combining data-driven low-code programming with other approaches:** Combining our approach with other existing approaches, such as *output-directed-programming*[8], could improve the functionality of the overall system.

7. **Performance optimization:** Further research into optimizing the rendering and update algorithms could improve the system's performance with large datasets and complex applications.

These areas of future work aim to address the current limitations of the system while also exploring new possibilities opened up by this data-driven, low-code approach.

Conclusion

Bibliography

1. *React* [online]. [visited on 2024-04-03]. Available from: https://react.dev/.
2. *Vue.js* [online]. [visited on 2024-04-03]. Available from: https://vuejs.org/.
3. Pinho, Daniel; Aguiar, Ademar; Amaral, Vasco. What about the usability in low-code platforms? A systematic literature review. *Journal of Computer Languages*. 2023, vol. 74, p. 101185. ISSN 25901184. Available from DOI: 10.1016/j.cola.2022.101185.
4. Sahay, Apurvanand; Indamutsa, Arsene; Di Ruscio, Davide; Pierantonio, Alfonso. Supporting the understanding and comparison of low-code development platforms. In: *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Portoroz, Slovenia: IEEE, 2020, pp. 171–178. ISBN 9781728195322. Available from DOI: 10.1109/SEAA51224.2020.00036.
5. *Mendix* [online]. [visited on 2024-05-16]. Available from: https://www.mendix.com.
6. *Darklang* [online]. [visited on 2024-04-22]. Available from: https://darklang.com.
7. Hempel, Brian; Lubin, Justin; Chugh, Ravi. Sketch-n-Sketch: Output-Directed Programming for SVG. In: *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*. New Orleans LA USA: ACM, 2019, pp. 281–292. ISBN 9781450368162. Available from DOI: 10.1145/3332165.3347925.
8. Chugh, Ravi; Hempel, Brian; Spradlin, Mitchell; Albers, Jacob. Programmatic and direct manipulation, together at last. *ACM SIGPLAN Notices*. 2016, vol. 51, no. 6, pp. 341–354. ISSN 0362-1340. Available from DOI: 10.1145/2980983.2908103.
9. *Elm* [online]. [visited on 2024-04-29]. Available from: https://elm-lang.org.
10. *Elmish* [online]. [visited on 2024-06-11]. Available from: https://elmish.github.io.
11. *TodoMVC* [online]. [N.d.]. [visited on 2024-11-18]. Available from: http://todomvc.com.
12. *7GUIs benchmark* [online]. [visited on 2024-04-11]. Available from: https://eugenkiss.github.io/7guis.
13. *JetBrains Rider* [online]. [visited on 2024-11-23]. Available from: https://www.jetbrains.com/rider/.
14. *Github Copilot* [online]. 2024. [visited on 2024-11-23]. Available from: https://github.com/features/copilot.
15. *Visual Studio Code* [online]. [visited on 2024-11-23]. Available from: https://code.visualstudio.com/.
16. *Hugo*. Available also from: https://gohugo.io/.
17. Leonard, Peter. *Macworld Expo 1987 Boston*. 1987. Available also from: http://32by32.com/macworld-expo-1987-boston/.
18. Goodman, Danny. *The complete HyperCard handbook*. 2nd ed. Toronto ; New York: Bantam Books, 1988. The Macintosh performance library. ISBN 9780553345773.

19. Schreiber, Robin; Krahn, Robert; Ingalls, Daniel H. H.; Hirschfeld, Robert. *Transmorphic: Mapping direct manipulation to source code transformations.* Potsdam: Universitätsverlag, 2017. Technische Berichte des Hasso-Plattner-Instituts für Softwaresystemtechnik an der Universität Potsdam. isbn 9783869563879.

20. *F#* [online]. [visited on 2024-06-11]. Available from: https://fsharp.org.

21. *Fable* [online]. [visited on 2024-04-29]. Available from: https://fable.io.

22. *Feliz* [online]. [visited on 2024-06-11]. Available from: https://zaid-ajaj.github.io/Feliz.

23. Bray, Tim. *The JavaScript Object Notation (JSON) Data Interchange Format* [RFC 8259]. RFC Editor, 2017. Request for Comments, no. 8259. Available from doi: 10.17487/RFC8259.

24. *Fable.SimpleJson* [online]. [visited on 2024-06-11]. Available from: https://github.com/Zaid-Ajaj/Fable.SimpleJson.

25. Czaplicki, Evan; Chong, Stephen. Asynchronous functional reactive programming for GUIs. *ACM SIGPLAN Notices.* 2013, vol. 48, no. 6, pp. 411–422. issn 0362-1340, issn 1558-1160. Available from doi: 10.1145/2499370.2462161.

26. Elliott, Conal; Hudak, Paul. Functional reactive animation. In: *Proceedings of the second ACM SIGPLAN international conference on Functional programming.* New York, NY, USA: Association for Computing Machinery, 1997, pp. 263–273. ICFP '97. isbn 9780897919180. Available from doi: 10.1145/258948.258973.

27. Czaplicki. *A Farewell to FRP.* 2024. Available also from: https://web.archive.org/web/20241220190908/https://elm-lang.org/news/farewell-to-frp.

28. *Fable compatibility* [online]. [visited on 2024-04-29]. Available from: https://fable.io/docs/javascript/compatibility.html.

29. *GraphQL* [online]. [visited on 2024-05-16]. Available from: https://graphql.org.

30. Yang, Fan; Gupta, Nitin; Botev, Chavdar; Churchill, Elizabeth F; Levchenko, George; Shanmugasundaram, Jayavel. WYSIWYG development of data driven web applications. *Proceedings of the VLDB Endowment.* 2008, vol. 1, no. 1, pp. 163–175. issn 2150-8097. Available from doi: 10.14778/1453856.1453879.

31. *SAFE stack* [online]. [visited on 2024-04-03]. Available from: https://safe-stack.github.io/.

32. *Thoth.Json* [online]. [visited on 2024-06-12]. Available from: https://thoth-org.github.io/Thoth.Json/.

33. *Fable.Mocha* [online]. [visited on 2024-06-14]. Available from: https://github.com/Zaid-Ajaj/Fable.Mocha.

34. Park, Robert E. *Software Size Measurement: A Framework for Counting Source Statements:* Fort Belvoir, VA, 1992. Available from doi: 10.21236/ADA258304.

35. *7GUIs counter Example implementation* [online]. [visited on 2024-06-23]. Available from: https://github.com/eugenkiss/7guis-React-TypeScript-MobX/blob/master/src/app/guis/counter.tsx.

List of Figures

1 InterfaceSmith's UI example 8

1.1 Selection of a user level in the Hypercard preferences menu. 11
1.2 Example of Darklang's drag-and-drop user interface. 12
1.3 Example of Sketch-and-Sketch's user interface. 13
1.4 Example of the available JSON data types and the JSON syntax 14
1.5 Diagram of the Elmish Architecture 15

3.1 Single RenderingCode Creation Process 27

4.1 Core Logic Module Structure 30
4.2 Editor Module Structure 31
4.3 Example of *Data-driven UI's* user interface 33